

iMPS

Intelligent Maintenance Platform as a Service

SOURCE CODE USER MANUAL

คู่มือการใช้งาน Source Code สำหรับนักพัฒนา

สารบัญ

- บทที่ 1: ภาพรวมของระบบ (System Overview)
- บทที่ 2: ความต้องการของระบบ (System Requirements)
- บทที่ 3: การติดตั้งและตั้งค่า (Installation & Setup)
- บทที่ 4: โครงสร้าง Source Code (Source Code Structure)
- บทที่ 5: การใช้งาน API (API Usage)
- บทที่ 6: การ Deploy ระบบ (Deployment Guide)
- บทที่ 7: การแก้ไขปัญหาเบื้องต้น (Troubleshooting)
- บทที่ 8: ข้อมูลอ้างอิง (Reference)

บทที่ 1 : ภาพรวมของระบบ (System Overview)

1.1 iMPS คืออะไร?

iMPS (EV Charging Station Intelligent Maintenance Platform as a Service) คือแพลตฟอร์มอัจฉริยะที่การไฟฟ้าฝ่ายผลิตแห่งประเทศไทย (กฟผ.) พัฒนาขึ้นเพื่อบริหารจัดการและบำรุงรักษาสถานีอัดประจุไฟฟ้าเชิงป้องกัน โดยใช้เทคโนโลยี AI และ Data Analytics

1.2 Tech Stack

ส่วนงาน	เทคโนโลยี	หน้าที่
Frontend + Backend	Next.js + TypeScript	หน้าเว็บและ API
Data Pipeline	Python	ประมวลผลและวิเคราะห์ข้อมูล
UI Framework	Tailwind CSS	จัดสไตล์หน้าเว็บ
Version Control	Git + GitHub	เก็บและจัดการ Source Code
Code Editor	VS Code	เครื่องมือเขียน Code

บทที่ 2: การติดตั้งและตั้งค่า (Installation & Setup)

ขั้นตอนที่ 1: ติดตั้ง Git

- 1.1 เปิดเบราว์เซอร์ไปที่ → <https://git-scm.com/download/win>
- 1.2 คลิก "Click here to download" — ไฟล์จะโหลดอัตโนมัติ
- 1.3 เปิดไฟล์ .exe ที่โหลดมา แล้วกด Next ไปเรื่อยๆ จนถึง Install (ใช้ค่า default ได้เลย)
- 1.4 กด Finish
- 1.5 ตรวจสอบว่าติดตั้งสำเร็จ → เปิด Command Prompt (กด Win + R พิมพ์ cmd แล้ว Enter) แล้วพิมพ์:

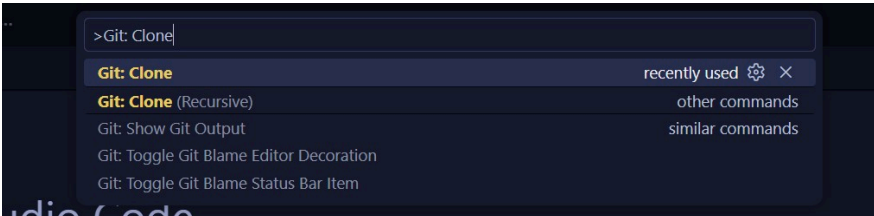
```
git --version
```

ถ้าขึ้นแบบนี้แสดงว่าสำเร็จ

```
git version 2.x.x.windows.x
```

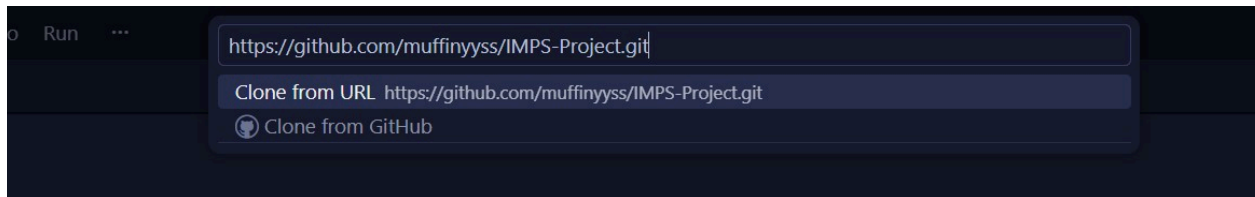
ขั้นตอนที่ 2: เปิด VS Code แล้ว Clone โปรเจค

- 2.1 เปิด VS Code
- 2.2 กด Ctrl + Shift + P เพื่อเปิด Command Palette
- 2.3 พิมพ์ว่า Git: Clone แล้วกด Enter

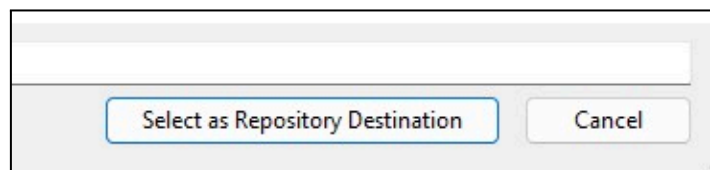
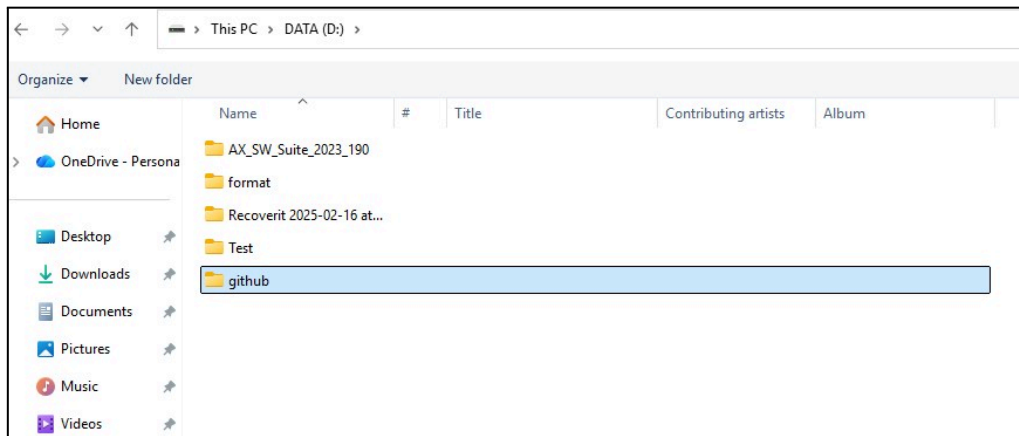


2.4 วาง URL นี้ลงในช่องที่ขึ้นมา แล้วกด Enter:

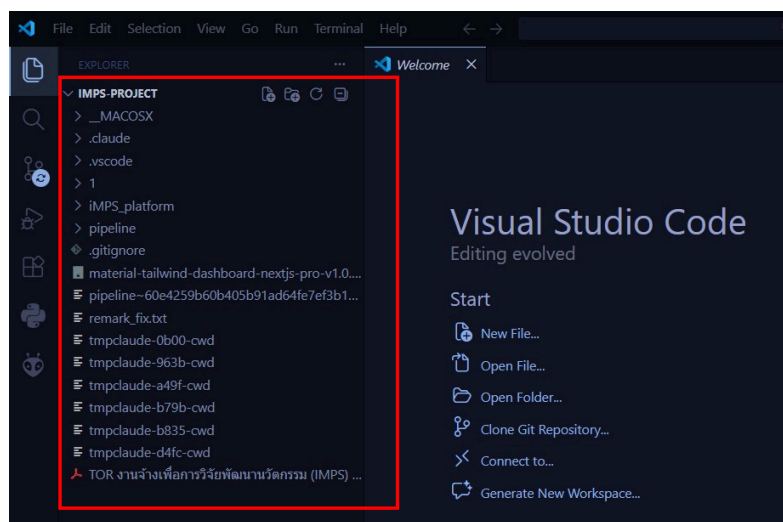
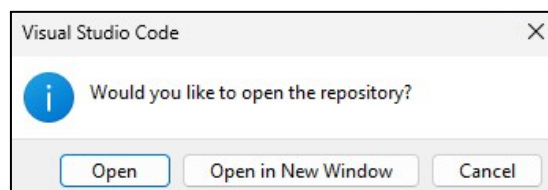
```
https://github.com/muffinyyss/IMPS-Project.git
```



2.5 เลือก folder ที่ต้องการเก็บโปรเจกต์ เช่น Desktop หรือ Documents



2.6 รอสักครู่... พอโหลดเสร็จจะมีกล่องถามที่มุมขวาล่างว่า "Open Repository?" → กด Open



ขั้นตอนที่ 3: ติดตั้งและรัน Backend (Python)

3.1 ไปที่ → <https://www.python.org/downloads/>

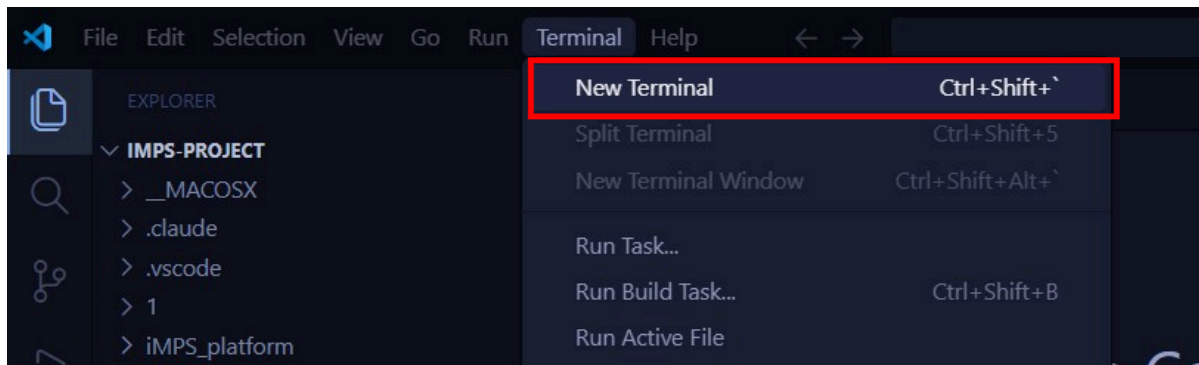
3.2 กด "Download Python 3.x.x" แล้วติดตั้ง

3.3 สำคัญ ตอนติดตั้งต้องติ๊ก "Add Python to PATH" ด้วย

3.4 ตรวจสอบโดยเปิด Terminal ใน VS Code (Ctrl+`) แล้วพิมพ์:

```
python --version
```

3.5 เข้า Terminal



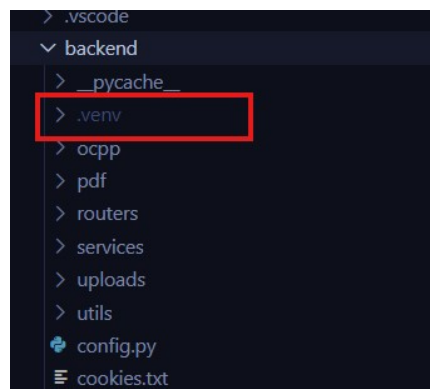
3.6 เข้าไปใน folder backend

```
cd backend
```

3.7 สร้าง Virtual Environment (.venv)

```
python -m venv .venv
```

รอสักครู่... จะเห็น folder .venv โผล่ขึ้นมาใน backend



3.8 Activate .venv

```
.venv\Scripts\activate
...
พอสำเร็จจะเห็น ** (.venv) ** ขึ้นหน้า path แบบนี้:
...
(.venv) PS C:\...\backend>
```

3.9 ติดตั้ง dependencies จาก requirements.txt

```
pip install -r requirements.txt
```

```
, setuptools, python-multipart, python-dotenv, pyphen, PyJWT, pydyf, pycparser, pyasn1, pillow, paho-mqtt, numpy, MarkupSafe, invoke, idna, h11, fonttools, dnspyt
hon, defusedxml, colorama, charset-normalizer, certifi, bcrypt, asgiref, annotated-types, aiomysql, typing-inspection, rsa, requests, python-dateutil, pymongo,
pydantic_core, Jinja2, fpdf2, email-validator, ecdsa, Django, cssselect2, click, cffi, anyio, weasyprint, uvicorn, starlette, python-jose, PyNaCl, pydantic, panda
s, motor, cryptography, paramiko, fastapi
Successfully installed Brotli-1.1.0 Django-5.2.5 Jinja2-3.1.6 MarkupSafe-3.0.3 PyJWT-2.10.1 PyNaCl-1.5.0 aiomysql-5.0.0 annotated-types-0.7.0 anyio-4.10.0 asgiref-3.9.1
bcrypt-4.3.0 certifi-2025.8.3 cffi-1.17.1 charset-normalizer-3.4.3 click-8.2.1 colorama-0.4.6 cryptography-45.0.6 cssselect2-0.8.0 defusedxml-0.7.1 dnspython-2.7.0
ecdsa-0.19.1 email-validator-2.3.0 fastapi-0.116.1 fonttools-4.59.2 fpdf2-2.8.5 h11-0.16.0 idna-3.10 invoke-2.2.0 motor-3.7.1 numpy-2.3.2 paho-mqtt-2.1.0 pandas-2.3.2
paramiko-4.0.0 passlib-1.7.4 pillow-11.3.0 pyasn1-0.6.1 pycparser-2.22 pydantic-2.11.7 pydantic_core-2.33.2 pydyf-0.11.0 pymongo-4.14.0 pyphen-0.17.2 python-dateutil-2.9.0
post0 python-dotenv-1.1.1 python-jose-3.5.0 python-multipart-0.0.20 pytz-2025.2 requests-2.32.5 rsa-4.9.1 setuptools-80.9.0 six-1.17.0 sniffio-1.3.1 sqlparse-0.5.3
starlette-0.47.2 tinycss2-1.4.0 tinyhtml5-2.0.0 typing-inspection-0.4.1 typing_extensions-4.14.1 tzdata-2025.2 urllib3-2.5.0 uvicorn-0.35.0 weasyprint-66.0
webencodings-0.5.1 wheel-0.45.1 zopfli-0.2.3.post1

[notice] A new release of pip is available: 24.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) D:\github\IMPS-Project\iMPS_platform\backend>
```

3.10 รัน backend

```
uvicorn main:app --reload
```

ขั้นตอนที่ 4: ติดตั้งและรัน Frontend (React/Next.js)

4.1 ไปที่ → <https://nodejs.org/>

4.2 กด "LTS" แล้วติดตั้ง (กด Next ไปเรื่อยๆ)

4.3 ตรวจสอบโดยพิมพ์ใน Terminal:

```
node --version
npm --version
```

```
(.venv) D:\github\IMPS-Project\iMPS_platform\backend>node --version
v22.11.0
```

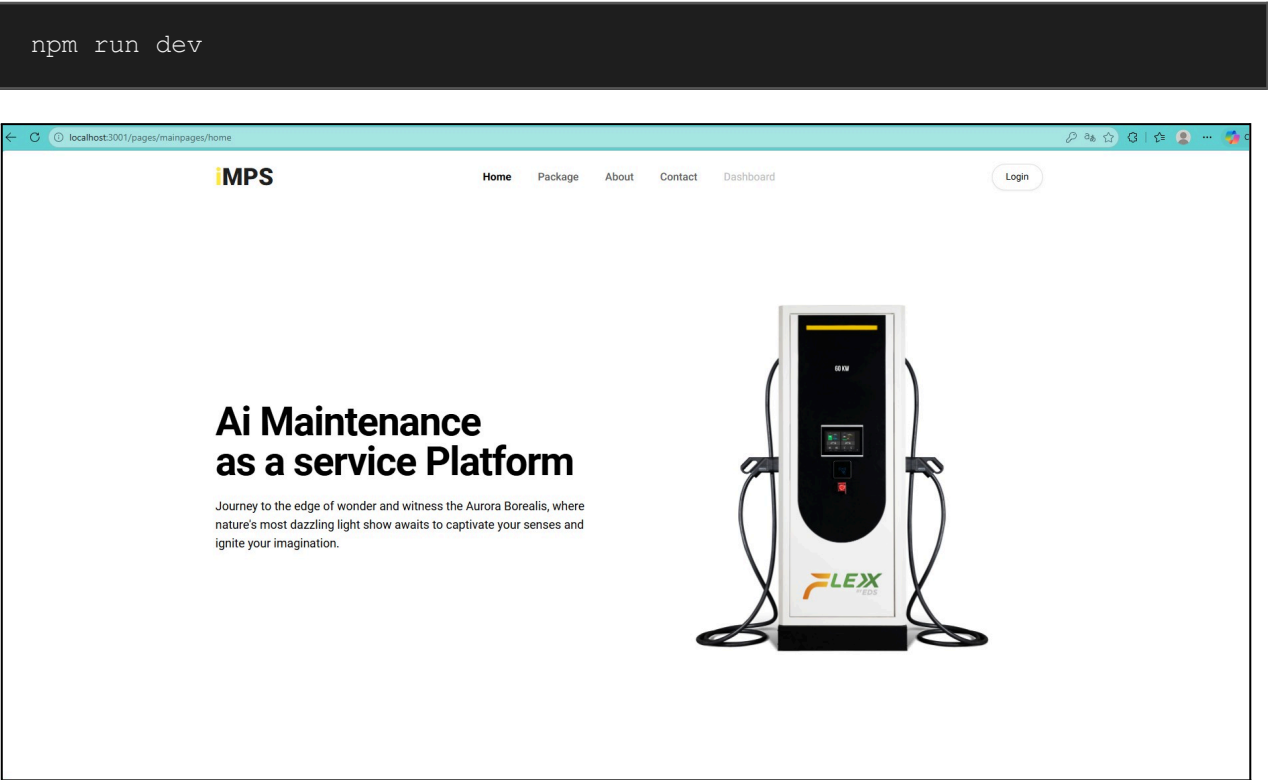
```
(.venv) D:\github\IMPS-Project\iMPS_platform\backend>npm --version
11.1.0
```

4.3 เปิดTerminal ใหม่

```
npm install
```

รอจนเสร็จ จะเห็น folder node_modules โผล่ขึ้นมา

4.4 รัน Frontend



บทที่ 3: โครงสร้าง Source Code (Source Code Structure)

3.1 โครงสร้างโฟลเดอร์ระดับ Root

Path / File	คำอธิบาย
.env	ตัวแปรสภาพแวดล้อม (API keys, DB, secrets)
next.config.js	การตั้งค่า Next.js framework
package.json	Dependencies และ scripts ของ Node.js
tailwind.config.js	การตั้งค่า TypeScript compiler
tsconfig.json	การตั้งค่า TypeScript compiler
requirements.txt	Python dependencies (ระดับ root)

3.2 Backend (backend/)

3.2.1 ไฟล์หลัก

Path / File	คำอธิบาย
.main.py	main.py จุดเริ่มต้น FastAPI ลงทะเบียน router ทั้งหมด เริ่ม Uvicorn
config.py	การตั้งค่าระบบ (env vars, API URLs, secrets)
deps.py	Dependencies รวม (เชื่อมต่อ DB, ตรวจสอบสิทธิ์)
requirements.txt	รายการ package Python

3.2.2 Routers (API Endpoints)

Path / File	คำอธิบาย
users.py	ยืนยันตัวตน, สมัครสมาชิก, โปรไฟล์
stations.py	CRUD สถานีชาร์จ
device.py	จัดการอุปกรณ์เครื่องชาร์จ
setting.py	ตั้งค่าระบบและเครื่องชาร์จ
cbm.py	Condition-Based Monitoring
mdb.py	ข้อมูล Main Distribution Board
ai.py	AI/ML วิเคราะห์และทำนายผล
notifications.py	ระบบแจ้งเตือน
auto_cm_watcher.py	เฝ้าระวัง CM อัตโนมัติ
cmreport.py	CRUD รายงาน Corrective Maintenance
pmreport_*.py	รายงาน PM (charger/station/mdb/cbbox/ccb)
pm_helpers.py	ฟังก์ชันช่วยใช้ร่วมกันใน PM
testreport_ac.py	รายงานทดสอบ AC
testreport_dc.py	รายงานทดสอบ DC

3.2.3 Services / PDF

Path / File	คำอธิบาย
services/maximo.py	IBM Maximo API client ทรัพย์สินและใบสั่งงาน
pdf/pdf_routes1.py	API สร้างและดาวน์โหลด PDF
pdf/templates/pdf_*.py	แม่แบบ PDF แต่ละประเภทรายงาน
pdf/fonts/	ฟอนต์ไทย TH Sarabun New

3.3 Frontend (src/)

3.3.1 ไฟล์หลัก

Path / File	คำอธิบาย
src/routes.jsx	กำหนดเส้นทางและเมนู
src/app/layout.tsx	Root layout (HTML head, providers, ฟอนต์)
src/app/page.tsx	หน้าแรก (redirect ไป dashboard)
src/app/globals.css	CSS รวมและ Tailwind imports

3.3.2 Dashboard

Path / File	คำอธิบาย
dashboard/analytics/	หน้าภาพรวม กราฟและสถิติ
dashboard/stations/	จัดการสถานี (เพิ่ม, แก้ไข, ลบ)
dashboard/chargers/	รายละเอียดเครื่องชาร์จ (Health Index, AI)
dashboard/device/	Dashboard DC charger Real-time
dashboard/cbm/	Condition-Based Monitoring
dashboard/setting/	ตั้งค่าเครื่องชาร์จ
dashboard/ai/	AI วิเคราะห์ + [moduleId]
dashboard/users/	จัดการผู้ใช้
dashboard/notifications/	ศูนย์แจ้งเตือน
dashboard/pm-report/	PM: charger, station, mdb, cb-box, ccb
dashboard/cm-report/	CM: open, inprogress, closed
dashboard/test-report/	Test: ac, dc

3.2.3 Shared Code

Path / File	คำอธิบาย
src/components/	ThemeProvider, MaterialTailwind, SiteNavbar, SiteFooter
src/context/index.tsx	React Context state รวม
src/services/	charging-api.ts (Axios), socket-service.ts (Socket.IO)
src/data/	ข้อมูลคงที่/จำลอง
src/utils/	api.ts, format helpers, useLanguage hook
src/theme/	Material Tailwind style overrides
src/widgets/	UI ซ้ำ: layout, cards, charts, tables

3.4 รายละเอียด Source Code ระดับไฟล์

ส่วนนี้อธิบายระดับ source code ว่าแต่ละไฟล์ทำหน้าที่อะไร แต่ละส่วนทำงานยังไง และเชื่อมต่อกับไฟล์อื่นอย่างไร ใช้ศัพท์ dev ตรงๆ

backend/main.py

Application Entry Point — Bootstrap FastAPI, register middleware, mount routers, spawn background tasks

Lifespan (Startup / Shutdown)

- lifespan() เป็น async context manager ที่ FastAPI เรียกตอน startup/shutdown
- Startup: inject errorDB + mongo_client เข้า app.state, spawn email_watcher เป็น asyncio.Task, เรียก start_watcher()
- Shutdown: cancel email task + stop_watcher()

Email Watcher (Background Task)

- Infinite loop, poll ทุก 1 วินาที
- รวบรวม collection ใน errorDB (FaultStatus) → เช็ค document ใหม่โดยเทียบ _id กับ last_ids
- ถ้าเจอ fault ใหม่ → resolve station/charger info (cache ไว้ใน sn_cache)
- เรียก parse_fault_message() → สร้าง notification dict
- _send_email_for_notification() → query email_notification_rules → match station + type → resolve user emails → send_email_smtp()

CORS Middleware

- Whitelist: localhost:3000/3001 (dev) + imps.egatdiamond.co.th (prod)
- allow_credentials=True เพราะ auth ใช้ cookie-based JWT

Note:

ถ้า Frontend deploy URL ใหม่ ต้องเพิ่ม origin ใน allow_origins ไม่งั้น CORS error

Static Files + Router Registration

- Mount /uploads → serve รูปที่ upload (PM/CM/test reports)
- Register 16 routers ด้วย app.include_router()
- PDF routes wrap ใน try/except เพราะ WeasyPrint อาจไม่มีบาง env

backend/config.py

Central Configuration — เก็บทุก connection, credential, constant, helper ที่ modules อื่น import มาใช้

MongoDB Connections

- client = AsyncIOMotorClient — async driver ใช้กับ routers ทั้งหมด
- client1 = MongoClient (sync) — ใช้เฉพาะ PDF generation

Database references หลัก:

Path / File	คำอธิบาย
errorDB	client["FaultStatus"] — fault/error จากเครื่องชาร์จ
deviceDB	client["utilizationFactor"] — utilization data
settingDB	client["settingParameter"] — ค่าตั้ง charger
CBM_DB	client["monitorCBM"] — Condition-Based Monitoring
MDB_DB / MDB_realtime_DB	Main Distribution Board + realtime
PMReportDB / CMReportDB	PM และ CM reports + URL collections
DCTestReportDB / ACTestReportDB	Test reports AC/DC
OUTPUT_DBS	dict 7 AI output databases (module1–module7)
INPUT_DBS	dict 7 AI input/prediction databases

JWT / Auth Config

- SECRET_KEY = "supersecret", ALGORITHM = "HS256"
- Token expiry: 1440 min (24 ชั่วโมง), Session idle: technician = unlimited, default = 15 min
- create_access_token(data, expires_delta) — encode payload + exp claim ด้วย python-jose

MQTT Client

- Connect broker 203.154.130.132:1883 topic imps/setting — publish ค่าส่งไป edge box/charger

SMTP Config

- Gmail SMTP smtp.gmail.com:587 — สำหรับ email notifications, credentials อ่านจาก env vars

Helper Functions

Path / File	คำอธิบาย
_validate_station_id()	เช็ค regex, raise 400 ถ้า invalid
get_mdb_collection_for(sid)	return dynamic collection จาก station_id
to_json(obj)	serialize dict → JSON (handle ObjectId, Decimal128, datetime)
create_access_token()	encode JWT payload + exp claim
parse_iso_dt() / _to_utc_dt()	parse ISO string → datetime object
to_float(x)	safe cast float, handle Decimal128/None/string
normalize_pm_date(s)	ISO/UTC → YYYY-MM-DD ตาม Asia/Bangkok
floor_bin(dt, step_sec)	floor datetime ลง bin size สำหรับ aggregate time-series

UserClaims (Pydantic BaseModel)
Decoded token payload ที่ router ได้รับหลังเรียก Depends():

Path / File	คำอธิบาย
sub: str	subject — email ของ user
user_id: Optional[str]	MongoDB _id as string
role: str	"admin" "technician" "user"
station_ids: List[str]	list station ที่ user มีสิทธิ์ access
company: Optional[str]	company name

get_current_user(request) → UserClaims
Token resolution order:
1. ดึงจาก cookie access_token
2. ถ้าไม่มี → header Authorization: Bearer ...
3. ถ้าไม่มีทั้งคู่ → 401 Not authenticated

Decode + Error:

- ExpiredSignatureError → 401 token_expired
- JWTError → 401 invalid_token

วิธีใช้ใน router:

```
@router.get("/api/something")
async def something(user: UserClaims = Depends(get_current_user)):
    # user.role, user.station_ids พร้อมใช้
```

Dependency Graph
config.py (connections, constants, helpers) ← main.py (bootstrap, register routers) ← deps.py (auth) ← ทุก router เรียก Depends()

src/routes.jsx

Sidebar Menu + Role-Based Access Control — กำหนดเมนู Sidebar, ควบคุม role access, ซ่อน/แสดงเมนูตามสถานะการเลือก charger

baseRoutes — Menu Definition Array

เมนูทั้งหมดประกาศไว้ใน array แต่ละ item มี property:

- name — ชื่อแสดงบน Sidebar
- path — URL ปลายทาง
- allow — array ของ role ที่เห็นเมนูนี้ เช่น ["admin","owner"] หรือ ["*"] = ทุก role
- showMode — "before" = แสดงก่อนเลือก charger, "after" = หลังเลือก, "both" = แสดงเสมอ
- pages — sub-menu (optional) เช่น Profile, Help, Logout
- external + newTab — ลิงก์ภายนอก เช่น Ai Module ชี้ไป :8001

เมนูทั้งหมดที่ประกาศ:

เมนู	path	allow	showMode
(ชื่อ user)	sub: profile, help, logout	admin, owner, technician	both
Stations	/dashboard/stations	admin, owner, technician	both
Users	/dashboard/users	admin เท่านั้น	before
My Charger	/dashboard/chargers	admin, owner	after
Device	/dashboard/device	admin, owner	after
Configuration	/dashboard/setting	admin, owner	after
Condition-base	/dashboard/cbm	admin, owner	after
MDB/CCB	/dashboard/mdb	admin, owner	after
PM report	/dashboard/pm-report	admin, owner, technician	after
CM report	/dashboard/cm-report	admin, owner, technician	after
Test report	/dashboard/test-report	admin, owner, technician	after
Ai Module	external :8001	admin, owner	after

Key Functions

- readAuthFromStorage() — อ่าน user data จาก localStorage key "user" หรือ "auth" → return { username, email, role }
- canSee(allow, roles) — เช็ค role ของ user อยู่ใน allow array ไหม
- canSeeByMode(showMode, hasCharger) — เช็คตาม showMode vs สถานะการเลือก charger
- personalize(items) — เปลี่ยนชื่อ "admin" → username จริงของคนที login
- getRoutes(roles, hasCharger) — flow: baseRoutes → prune (filter role+mode) → personalize

useRoutes(rolesFromApp) — React Hook

- Track charger selection จาก: URL params (sn, station_id), localStorage (selected_sn), custom events, poll 500ms
- CBM toggle: listen event cbm:toggle ถ้า inactive → ซ่อนเมนู Condition-base
- Return filtered + personalized routes ผ่าน useMemo

Dev Note

เพิ่มเมนูใหม่ → เพิ่ม object ใน baseRoutes กำหนด allow + showMode | เปลี่ยนสิทธิ์ → แก้ allow array | เพิ่ม role ใหม่ → เพิ่มชื่อ role ใน allow ของเมนูที่ต้องการ

บทที่ 4: การใช้งาน API (API Usage)

4.1 Authentication

ระบบใช้ JWT (JSON Web Token) หลัง login สำเร็จจะได้ token กลับมา ต้องแนบใน Header ทุก request:

```
Authorization: Bearer <token>
```

4.2 API Endpoints หลัก

Users

Method	Endpoint	คำอธิบาย
POST	/api/auth/login	เข้าสู่ระบบ รับ JWT token
POST	/api/auth/register	สมัครสมาชิกใหม่
GET	/api/users	ดึงรายการผู้ใช้ทั้งหมด

Stations / Devices

Method	Endpoint	คำอธิบาย
GET	/api/stations	ดึงรายการสถานี
POST	/api/stations	เพิ่มสถานีใหม่
GET	/api/devices	ดึงรายการอุปกรณ์
GET	/api/cbm/{charger_id}	ข้อมูล CBM ของเครื่องชาร์จ

PM / CM / Test Report

Method	Endpoint	คำอธิบาย
GET	/api/pmreport/{type}	ดึงรายงาน PM ตามประเภท
POST	/api/pmreport/{type}	สร้างรายงาน PM
GET	/api/cmreport	ดึงรายงาน CM
POST	/api/cmreport	สร้างรายงาน CM
GET	/api/testreport/ac	รายงานทดสอบ AC
GET	/api/testreport/dc	รายงานทดสอบ DC
GET	/api/pdf/{type}/{id}	สร้างและดาวน์โหลด PDF

AI / Notifications

Method	Endpoint	คำอธิบาย
GET	/api/ai/predict/{module}	AI ทำนายผล
GET	/api/notifications	ดึงรายการแจ้งเตือน